

AI 인프라실 기술 보고서

JAMYPG NL2SQL MCP 사용자 가이드

자연어-SQL 변환 및 안전 실행을 위한 연동 가이드라인

작성 부서	AI 인프라실
발행 일자	2026년 7월 10일
문서 버전	v0.1.0
보안 구분	대외비 (Confidential)

보고 부서: AI 인프라실

작성 일자: 2026년 7월 8일

문서 버전: v1.1.0

1. 개요 및 핵심 개념

본 가이드는 JAMYPG NL2SQL MCP 서버를 활용하여 자연어 질문을 PostgreSQL·MySQL·MariaDB SQL로 안전하게 변환하고 실행하려는 개발자, 데이터 엔지니어 및 LLM 연동 담당자를 위해 작성되었습니다.

JAMYPG는 **Model Context Protocol (MCP)** 규격을 탑재하여 대화형 언어 모델(LLM)이 직접 사내 데이터에 접근하지 않고도, 가상의 카탈로그 정보와 정밀 정적 검증 도구를 경유하여 안전한 범위 내에서만 질의를 발행하도록 설계된 솔루션입니다.

최근 버전(v0.12.0+)부터는 단순 시드 매칭의 단계를 넘어 조인 토폴로지와 데이터 값 증거를 확장하는 **GraphRAG 기반 검색 계층(`retrieve_context`)** 및 사용자의 재질문 답변을 피드백 가중치로 자동 환류시키는 **재질문 학습 루프**가 탑재되어 연동 정확도가 더욱 향상되었습니다.

2. 권장 NL2SQL 연동 워크플로 (10단계)

자연어 질문을 받았을 때 클라이언트 에이전트와 LLM은 반드시 아래의 표준 **10단계 흐름**을 순서대로 이행해야 합니다. 이 순서를 건너뛰면 보안 컴플라이언스 위배 및 문법 오류 가능성이 매우 높아집니다.

단계	프로세스 및 호출 도구	실행 동작 및 조건 분기	산출물 및 다음 행동
Step 1	사용자 질문 접수	사용자가 자연어 질문을 에이전트에 입력	다음 단계로 질문 전달
Step 2	질문 그라운드링 및 GraphRAG <code>prepare_sql_context</code> 호출	질문 분석, GraphRAG (<code>retrieve_context</code> 내부 구동), 되묻기 항목 진단 일괄 가동	<code>status</code> 판정에 따라 분기 수행

단계	프로세스 및 호출 도구	실행 동작 및 조건 분기	산출물 및 다음 행동
Step 3	되물기 분기 게이트 및 환류	- [needs_clarification]: Blocking 모호성 감지 시 중단 및 사용자 재질문 - [ready]: 모호성 해결 완료 시 다음 단계로 진행	- [needs_clarification]: 답변 수집 후 Step 2 재진입 (※ 답변 해소 시 선택 테이블 corrected 피드백으로 자동 기록 및 사용량 가점 환류) - [ready]: 스켈레톤 및 그라운드링 정보 인계
Step 4	SQL 스켈레톤 획득	준비된 <code>skeleton.skeleton_sql</code> , 지표 식, 조인 경로 확보	SQL 골격 뼈대 획득
Step 5	SQL 조립 및 작성	LLM이 그라운드링 컨텍스트 내 컬럼정보만 사용하여 <code>/*</code> <code>SLOT */</code> 영역 조립	후보 SQL 완성
Step 6	정적 보안 검증 <code>validate_sql</code> 호출	SELECT 전용 검사, PII 민감 정보 누락, 33종 가드레일 가동	검증 결과 (<code>valid</code> 판단)
Step 7	자가 수정 피드백	- [valid == false]: 제공된 <code>fix_hints</code> 를 읽어 SQL 자동 보정 (최대 2회) - [valid == true]: 검증 통과 후 다음 단계 진행	- [valid == false]: 실패 사유 리턴 - [valid == true]: 검증 통과 SQL
Step 8	실행 계획 실측 <code>explain_sql</code> 호출	대상 DB에 연결하여 JSON EXPLAIN 기반으로 예상 Cost, Cartesian 조인, Full Scan 여부 분석	- [risk == high]: 조건식 보강 후 재생성 - [risk == low]: 다음 단계 진행
Step 9	안전 런타임 실행 <code>run_sql_safely</code> 호출	LIMIT 래핑, PII 값 마스킹 필터링, 결과 캐시 및 서킷 브레이커 가동	최종 데이터 셋 확보 및 사용자 표출

단계	프로세스 및 호출 도구	실행 동작 및 조건 분기	산출물 및 다음 행동
Step 10	피드백 이력 적재 record_feedback 호출	사용자 채택 여부(adopted) 및 오류 이력을 서버에 영속 적재	다음 컴파일 시 Few-shot 및 가중치 학습 반영

10단계 실행 프로세스 상세 명세

- 자연어 질의 수신:** 사용자가 대화창에 한글 또는 영어로 데이터 관련 요구사항을 작성합니다.
- prepare_sql_context 호출:** 질문 원문과 함께 본 도구를 호출합니다. 내부적으로 retrieve_context 가 자동 구동되어 시드 검색 테이블 외에 1-hop 조인 확장 테이블 및 값 일치 증거 테이블을 7대 가중치 신호에 맞추어 하이브리드 재정렬합니다.
- 되물기(Clarification) 판단 게이트:** 응답의 status 가 "needs_clarification" 인 경우, 절대로 SQL 작성을 개시하지 마십시오. clarifications 에 담긴 질문(선택지가 있을 경우 옵션 리스트와 권장 추천 배지)을 사용자 화면에 띄우고 답변을 받습니다. 사용자가 답을 주면 prepare_sql_context(question, clarifications={id: answer_or_option_key}) 로 다시 질의를 넣습니다. 이때 답변이 완료되면 해당 테이블 선택 사실이 outcome: "corrected" , source: "clarification" 피드백 로그로 즉시 자동 기록되어 usage prior 가점으로 학습됩니다.
- 골격 획득 및 컨텍스트 바인딩:** status 가 "ready" 로 떨어지면, 에이전트는 제공된 skeleton.skeleton_sql 조각과 schema_context 에 명시된 축소 스키마, metrics 사전의 지표 식을 수집합니다.
- 쿼리 조립 및 슬롯 매칭:** LLM은 스키텔론에 들어있는 /* SLOT: ... */ 주석 영역만 채우는 방식으로 최종 SQL 문을 작성합니다. 카탈로그에 없는 임의의 컬럼이나 조인을 개발해 내선 안 됩니다.
- validate_sql 호출:** 작성된 SQL 구문을 validate_sql 도구에 전달합니다. 이때 prepare_sql_context 번들에서 취득한 expected_output_columns 와 metric_names 를 각각 expected_outputs 및 metrics 파라미터로 동봉하여 전송합니다.
- 자가 수정 루프 (최대 2회):** 검증 에러(valid: false) 발생 시 응답 내의 fix_hints 를 읽어 들여 쿼리를 자가 수정하고 다시 validate_sql 을 호출합니다. 만약 2회 재시도 후에도 검증 실패 시, 쿼리 작성을 포기하고 실패 사유와 수정 대안을 사용자에게 출력합니다.
- explain_sql 성능 분석:** 가드레일을 통과한 SQL에 대해 explain_sql 을 가동해 예상 Cost와 실행 계획을 실측 검사합니다(PostgreSQL은 EXPLAIN (FORMAT JSON) , MySQL/MariaDB는 EXPLAIN FORMAT=JSON 활용). 만약 risk: "high" 가 감지되면 실행을 멈추고 suggestions에 따라 낱자 기간 조건이나 LIMIT 행 제한을 보강해 5단계로 돌아갑니다.

9. **run_sql_safely 실행**: 최종 조율된 SQL 문을 지정된 대상 DB 프로파일 ID(postgres/mysql/mariadb)로 구동하여 안전하게 행(Rows) 데이터를 확보합니다.
10. **record_feedback 적재**: 최종 쿼리의 성공/실패 여부, 사용자가 실제 채택하여 분석에 활용했는지 여부(`adopted: true`)를 피드백 로그로 적재하여 파이프라인을 종료합니다.

3. SQL 검증 룰 에러 해소 가이드 (비포/애프터 예시)

`validate_sql` 검증기가 감지하여 차단하는 33종의 룰 중 빈번하게 나타나는 핵심 이슈들의 수정 방법입니다.

3.1. [오류] PII_COLUMN (개인식별정보 노출 차단)

- **에러 상황**: overrides에 지정된 PII 속성 컬럼(예: 고객명 `cust_name`)을 SELECT 절에 가감 없이 기술한 경우.
 - **Before (에러 발생)**:

```
SELECT T1.cust_name, T1.credit_score
FROM dw_snapshot.customer_snapshot T1
LIMIT 100
```

- **After (해결 방법)**: PII 대상을 SELECT에서 완전히 빼거나, 부득이한 경우 마스킹 처리를 부과합니다.

```
SELECT SUBSTR(T1.cust_name, 1, 1) || '**' AS "고객명", T1.credit_score
FROM dw_snapshot.customer_snapshot T1
LIMIT 100
```

3.2. [오류] DIALECT_FUNCTION / DIALECT_LIMIT (대상 DB 방언 미준수)

- **에러 상황**: 오라클 전용 잔존 구문(`NVL` , `ROWNUM` , `FETCH FIRST` 등)을 사용하여 PostgreSQL·MySQL·MariaDB 방언 검증에 걸린 경우.
 - **Before (에러 발생)**:

```
SELECT T1.credit_grade, AVG(NVL(T1.credit_score, 0))
FROM dw_snapshot.customer_snapshot T1
GROUP BY T1.credit_grade
FETCH FIRST 50 ROWS ONLY
```

- **After (해결 방법):** NVL 은 표준 COALESCE 로, 행 제한은 대상 DB 공통 구문인 LIMIT n 으로 치환합니다 (fix_hints 에 치환 지침 제공: NVL→COALESCE, DECODE→CASE, ROWNUM→LIMIT).

```
SELECT T1.credit_grade, AVG(COALESCE(T1.credit_score, 0))
FROM dw_snapshot.customer_snapshot T1
GROUP BY T1.credit_grade
LIMIT 50
```

3.3. [경고] MISSING_DEL_FILTER / MISSING_EXCL_FILTER (정책 필터 누락)

- **예러 상황:** 카탈로그 데이터 모델 상 분석 대상 제외 코드나 삭제 데이터 제외 조건을 생략하여 통계 왜곡 위험이 높은 경우.
 - **Before (경고 발생):**

```
SELECT T1.credit_grade, AVG(T1.credit_score)
FROM dw_snapshot.customer_snapshot T1
WHERE T1.base_month = '202506'
GROUP BY T1.credit_grade
```

- **After (해결 방법):** 가이드라인에 명시된 기본 제외값 조건(excl_code_1 IS NULL 등)을 WHERE 절에 추가합니다.

```
SELECT T1.credit_grade, AVG(T1.credit_score)
FROM dw_snapshot.customer_snapshot T1
WHERE T1.base_month = '202506'
      AND T1.excl_code_1 IS NULL
GROUP BY T1.credit_grade
```

3.4. [오류] CODE_VALUE_UNKNOWN (비존재 공통 코드값 차단)

- **에러 상황:** 공통 코드 사전 원장에 존재하지 않는 임의의 비교값 리터럴을 조건절에 지정한 경우.
 - **Before (에러 발생):**

```
SELECT COUNT(*)
FROM dw_history.customers T1
WHERE T1.cust_type = '77' /* 존재하지 않는 고객구분코드 비교 */
```

- **After (해결 방법):** 검증기가 제공한 `fix_hints` 코드 대조표(예: '01':개인, '02':법인)를 확인하여 유효한 코드 리터럴만 매핑합니다.

```
SELECT COUNT(*)
FROM dw_history.customers T1
WHERE T1.cust_type = '01' /* 개인 고객 */
```

4. 실전 복잡 질의 SQL 생성 상세 예제

시나리오: 다중 조인 및 전월 대비 증감 현황 산출 (LAG 윈도우 함수 적용)

- **자연어 질문:** "dw_history 스키마의 고객정보와 카드계좌 테이블을 연결하여, 2025년 상반기 동안의 성별 카드 총이용금액을 월별로 집계하고 전월 대비 총이용금액의 증감분을 보여줘"
- `prepare_sql_context` 에서 추출된 정보:

- `selected_tables` : ["dw_history.customers", "dw_history.customer_transactions"]
- `join_paths` : `dw_history.customers.cust_id = dw_history.customer_transactions.cust_id` (INNER JOIN)
- `default_filters` : `dw_history.customer_transactions` 에 대해 `status_code IS NULL` 필수
- `time_conditions` : `dw_history.customer_transactions.trans_date BETWEEN '20250101' AND '20250630'`

• 최종 완성된 SQL (스켈레톤 구조를 엄격히 준수하여 조립):

```
WITH monthly_sales AS (
    SELECT T1.gender,
           SUBSTR(T2.trans_date, 1, 6) AS YYYYMM,
           SUM(T2.amount) AS TOT_AMT
    FROM dw_history.customers T1
    INNER JOIN dw_history.customer_transactions T2 ON T1.cust_id = T2.cust_id
    WHERE T2.trans_date ≥ '20250101' AND T2.trans_date ≤ '20250630'
           AND T2.status_code IS NULL
           AND T1.cust_type = '01' /* 개인 고객 한정 필터 */
    GROUP BY T1.gender, SUBSTR(T2.trans_date, 1, 6)
)
SELECT gender AS "성별코드",
       YYYYMM AS "이용연월",
       TOT_AMT AS "총이용금액",
       LAG(TOT_AMT, 1) OVER (PARTITION BY gender ORDER BY YYYYMM) AS "전월이용금",
       TOT_AMT - LAG(TOT_AMT, 1) OVER (PARTITION BY gender ORDER BY YYYYMM) AS "이번달이용금"
FROM monthly_sales
ORDER BY gender, YYYYMM
LIMIT 100
```

• 코드 리뷰 포인트:

- `cust_id` 키 컬럼을 기반으로 정확한 INNER JOIN 을 수립했습니다.

- `status_code IS NULL` 가이드 정책 필터를 빼놓지 않고 WHERE 조건에 추가해 데이터 왜곡을 방지했습니다.
- CTE(`WITH` 인라인뷰) 바깥에서 `LAG` 윈도우 함수를 정확히 호출하였으며, 전체 집계 결과가 쏟아지지 않도록 PostgreSQL·MySQL·MariaDB 공통 구문인 `LIMIT 100` 으로 행 개수를 안전하게 제한했습니다.